

# VK RPC roadmap

Ближайшее будущее

Григорий Бутейко vk:@hrissan

# VK RPC

Исторически развившийся протокол с историей

# VK RPC

Исторически развившийся протокол с историей

- Логический уровень
- Транспортный уровень

# VK RPC – логический уровень

Что есть

- Существуют соединения
- Query ID запросов в соединении для поиска ответов
- Запросы и ответы фиксированного размера (до 16MB)
- Заголовок для маршрутизации проксей
- Таймаут для каждого запроса
- Longpoll – просто отвечаем не сразу

# VK RPC – логический уровень

Что есть

- Существуют соединения
- Query ID запросов в соединении для поиска ответов
- Запросы и ответы фиксированного размера (до 16MB)
- Заголовок для маршрутизации проксей
- Таймаут для каждого запроса
- Longpoll – просто отвечаем не сразу
- Отмена запросов (как оптимизация)

# VK RPC – логический уровень

Чего нет

- Нет Ping – в транспорте
- Нет потоковой передачи (запросов/ответов большого размера)
- Нет серверных нотификаций

# VK RPC – транспортный уровень

Несколько транспортов

# VK RPC – транспортный уровень

Несколько транспортов

- VK TCP (PacketConn)
- VK UDP

# VK RPC – транспортный уровень

Несколько транспортов

- VK TCP (PacketConn)
- VK UDP
- Можно адаптировать к любому транспорту

# VK TCP (PacketConn)

Поверх stream сокета – TCP либо Unix

# VK TCP (PacketConn)

Поверх stream сокета – TCP либо Unix

В основном в пределах машины (но есть исключения)

# VK TCP (PacketConn)

что обеспечивает

- Границы сообщений
- Проверка живости (протокол пинг-понг)
- Шифрование

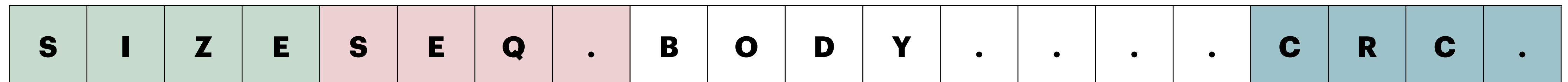
# VK TCP (PacketConn)

Границы сообщений

|          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| <b>S</b> | <b>I</b> | <b>Z</b> | <b>E</b> | <b>S</b> | <b>E</b> | <b>Q</b> | <b>.</b> | <b>B</b> | <b>O</b> | <b>D</b> | <b>Y</b> | <b>.</b> | <b>.</b> | <b>.</b> | <b>.</b> | <b>C</b> | <b>R</b> | <b>C</b> | <b>.</b> |
|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|

# VK TCP (PacketConn)

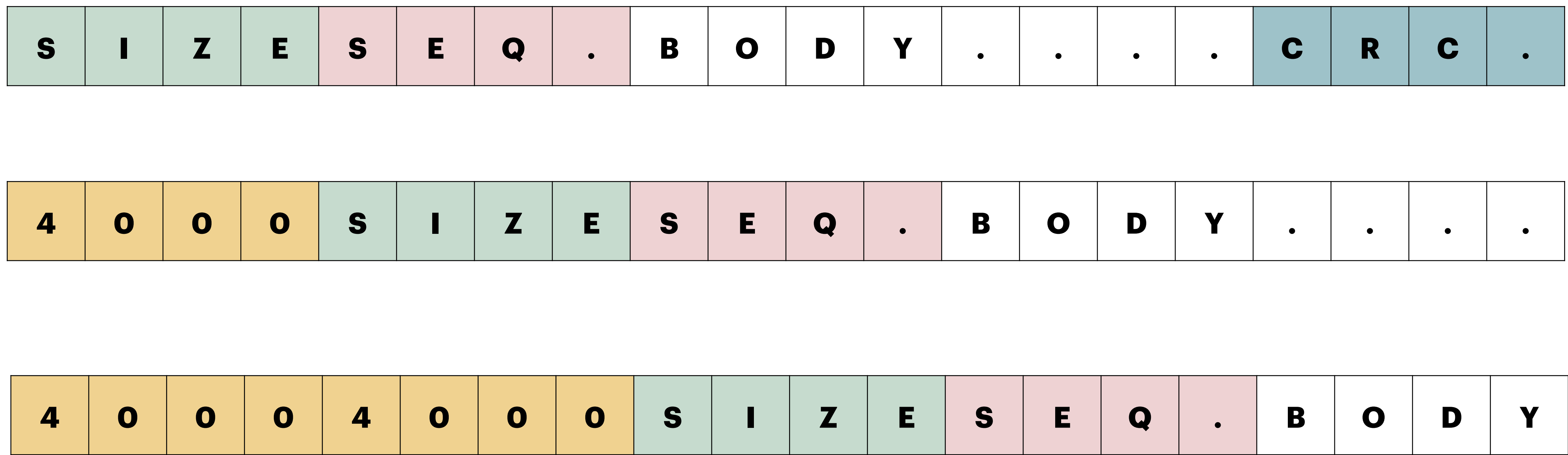
## Границы сообщений



- Все числа в LittleEndian
- Size всего пакета (но минимум 16 байт)
- Первые 4 байта body используются транспортом
- Seq не используется, просто увеличивается на 1
- CRC32 всего пакета до поля CRC, два вида

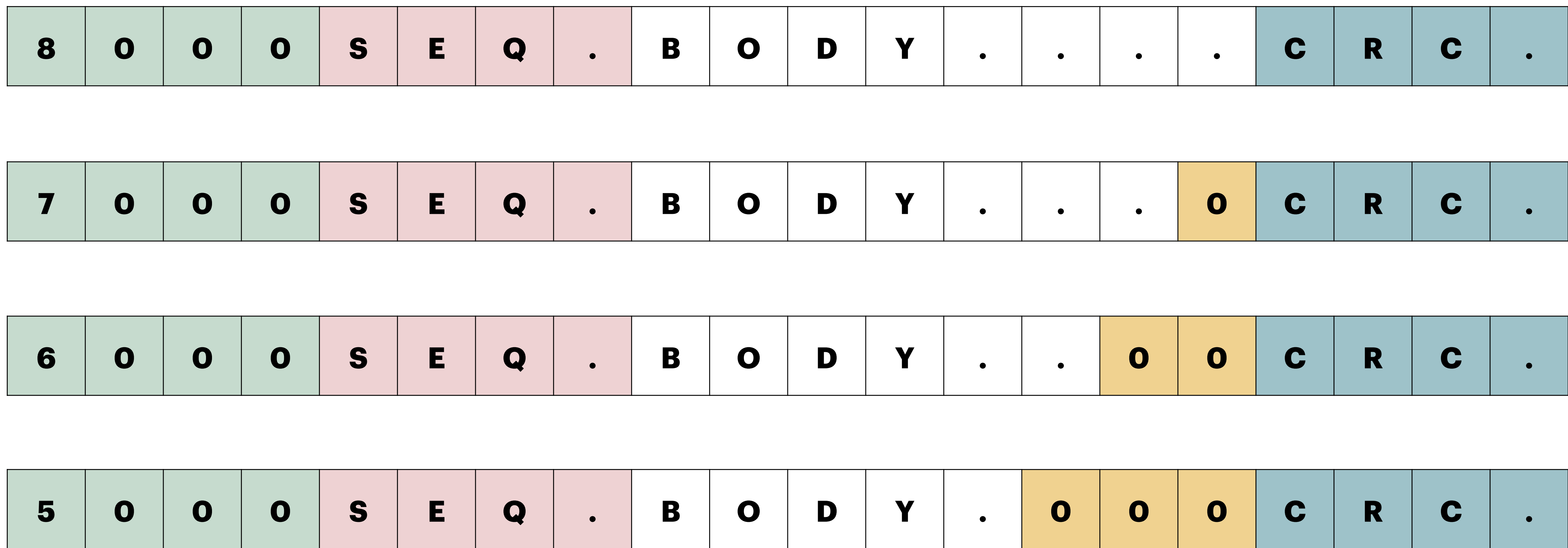
# VK TCP (PacketConn)

Границы сообщений – внешний padding четверками



# VK TCP (PacketConn)

Границы сообщений – внутренний padding до кратности 4



# VK TCP (PacketConn)

## ПИНГИ

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| S | I | Z | E | S | E | Q | . | P | i | n | g | . | . | . | . | . | . | . | . |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

- Пинг отправляется любой стороной, если долго не получала
- Нужно ответить пингом
- Если после отправки пинга долго нет ничего – разрыв сокета

# VK TCP (PacketConn)

Шифрование

# VK TCP (PacketConn)

## Шифрование

- При подключении обмен сообщениями Nonce, Handshake
- Нет forward secrecy (есть в Go реализации, скоро в C++)
- Неадекватная контрольная сумма (CRC32)
- Мало энтропии (16+16 байтов)
- Остальное в целом приемлемо

# VK TCP (PacketConn)

## Шифрование

- При подключении обмен сообщениями Nonce, Handshake
- Нет forward secrecy (есть в Go реализации, скоро в C++)
- Неадекватная контрольная сумма (CRC32)
- Мало энтропии (16+16 байтов)
- Остальное в целом приемлемо

Заменяем на TLS

# VK UDP

Поверх datagram сокетa – UDP

# VK UDP

Поверх datagram сокета – UDP

зачем, если есть более простой VK TCP?

# VK UDP

## Серьёзные недостатки VK TCP

- Head of line blocking
- Независимые потоки данных сделать невозможно
- Размер буферов сокетов (до 25 GB/100k соединений)
- Чтение и запись `syscall` на запрос/ответ если много соединений

# VK UDP

Что обеспечивает

- Идентификация, установка и разрыв соединений
- Границы сообщений
- Надежная доставка, подтверждения, переотправка
- Шифрование
- 0-RTT

# VK UDP

Что обеспечивает

- Идентификация, установка и разрыв соединений
- Границы сообщений
- Надежная доставка, подтверждения, переотправка
- Шифрование
- 0-RTT

Проверки живости нет,  $N^2$  ( $10^{10}$ ) пакетов/интервал

# VK UDP

Идентификация, установка и разрыв соединений

# VK UDP

Идентификация, установка и разрыв соединений

- NetPID – время и PID, клиент обеспечивает total order
- Connection ID – короткий (64 бит), единый для обеих сторон
- Клиентское и серверное соединение неотличимы

# VK UDP

Границы сообщений, надёжная доставка

# VK UDP

Границы сообщений, надёжная доставка

- Seqnum – неудобная пересборка
- Ошибки, как пользовательские сообщения

# VK UDP

## Шифрование

# VK UDP

## Шифрование

- Огромный заголовок (64+ байта?)
- Неадекватный CRC32
- При ротации ключей потери сообщений
- Нет forward secrecy
- Шаткая криптография

# VK UDP

## Шифрование

- Огромный заголовок (64+ байта?)
- Неадекватный CRC32
- При ротации ключей потери сообщений
- Нет forward secrecy
- Шаткая криптография

Заменяем на TLS?

# VK UDP

## Шифрование

- Огромный заголовок (64+ байта?)
- Неадекватный CRC32
- При ротации ключей потери сообщений
- Нет forward secrecy
- Шаткая криптография

Заменяем на DTLS

# DTLS

TLS для датаграмм

# DTLS

TLS для датаграмм

- Установка соединений (handshake)
- Шифрование и подписывание datagram
- Разрыв соединений

# DTLS

TLS для датаграмм

- Установка соединений (handshake)
- Шифрование и подписывание datagram
- Разрыв соединений

Нам понадобится протокол надежной доставки поверх

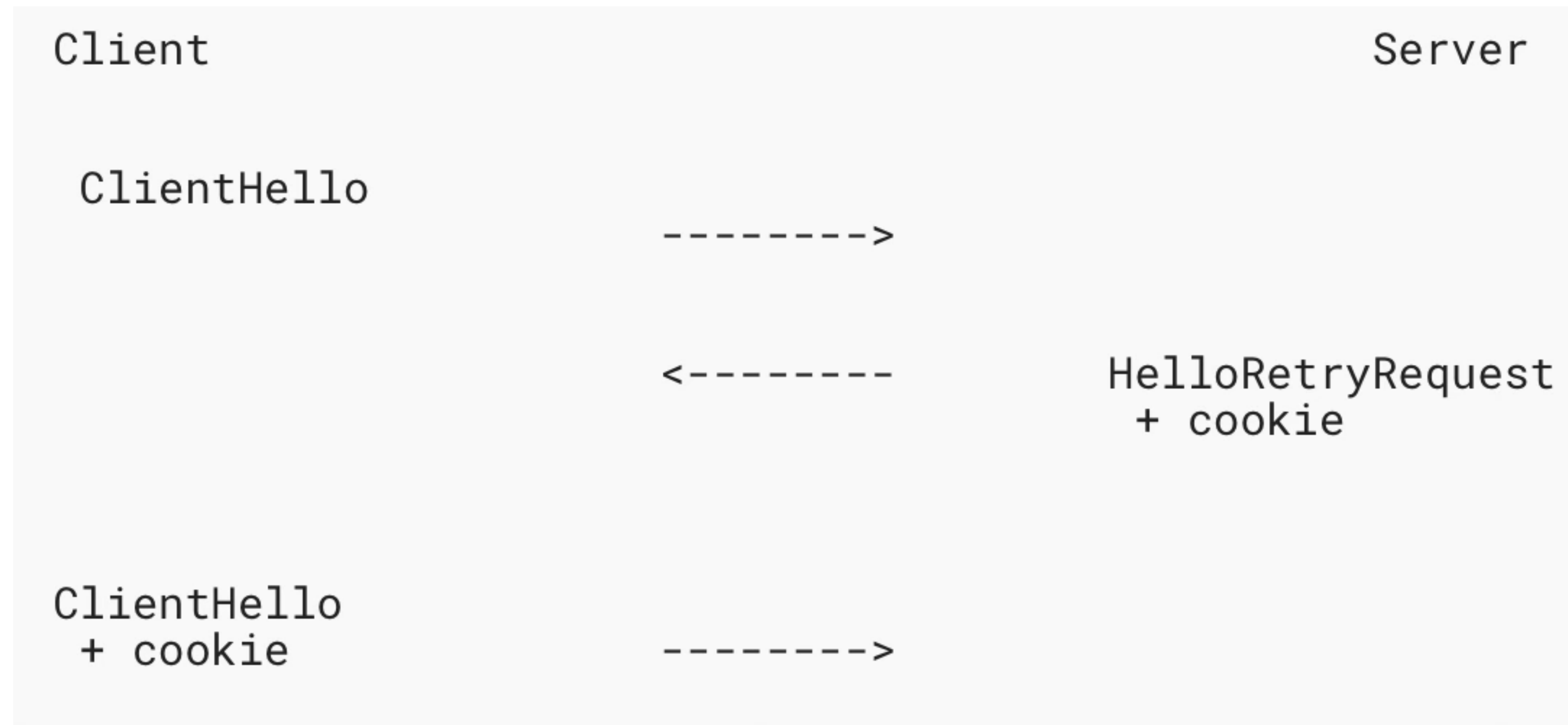
# DTLS handshake

Как в TLS, но сложнее из-за state machine надежной доставки

- С сертификатами X.509
- С PSK (pre shared key)

# DTLS handshake – reachability check

Чтобы не быть ретранслятором атаки, нужно проверить адрес клиента



# DTLS handshake с сертификатами

Идентификация сервера обязательна, клиента – по требованию сервера

```
ClientHello
+ cookie          ----->

                                     ServerHello
                                     {EncryptedExtensions}
                                     {CertificateRequest*}
                                     {Certificate*}
                                     {CertificateVerify*}
                                     {Finished}
<-----          [Application Data*]

{Certificate*}
{CertificateVerify*}
{Finished}
[Application Data] ----->

<-----          [ACK]
                  [Application Data*]
```

# DTLS handshake с PSK

## Идентификация взаимная с помощью PSK

```
ClientHello
+ pre_shared_key
+ psk_key_exchange_modes
+ key_share* ----->

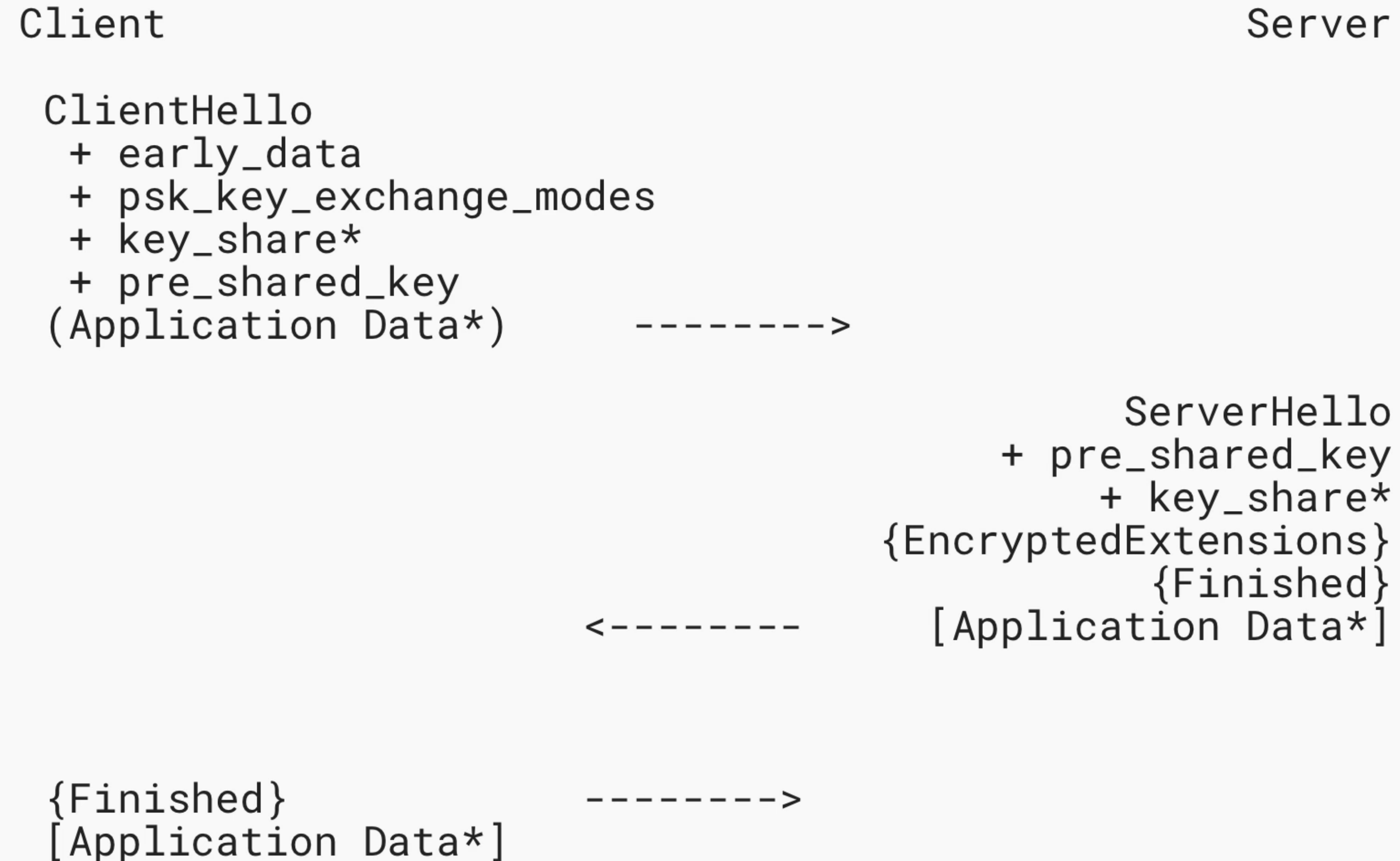
ServerHello
+ pre_shared_key
+ key_share*
{EncryptedExtensions}
{Finished}
[Application Data*]
<-----

{Finished} ----->
[Application Data*]

<----- [ACK]
[Application Data*]
```

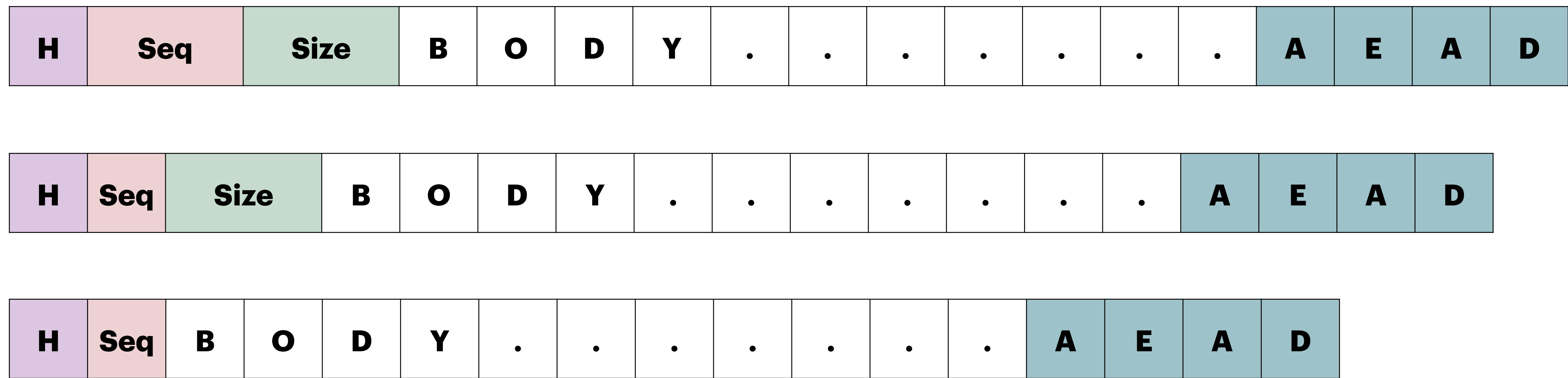
# DTLS handshake с PSK и 0-RTT

Данные в первом пакете, как и всегда, подвержены replay



# DTLS post handshake

Датаграмма состоит из последовательности записей (record)



# DTLS post handshake

Датаграмма состоит из последовательности записей (record)

|          |            |             |          |          |          |          |   |   |   |   |   |   |   |          |          |          |          |
|----------|------------|-------------|----------|----------|----------|----------|---|---|---|---|---|---|---|----------|----------|----------|----------|
| <b>H</b> | <b>Seq</b> | <b>Size</b> | <b>B</b> | <b>O</b> | <b>D</b> | <b>Y</b> | . | . | . | . | . | . | . | <b>A</b> | <b>E</b> | <b>A</b> | <b>D</b> |
|----------|------------|-------------|----------|----------|----------|----------|---|---|---|---|---|---|---|----------|----------|----------|----------|

- Заголовок всегда 1 байт, 2 бита эпохи
- Seq 1 или 2 байта
- Size (2 байта) может быть опущен для последней записи
- AEAD для популярных шифров 16 байтов
- Тело и Seq зашифровано, подписано, остальное подписано

# DTLS – ДОСТОИНСТВА

Весьма впечатляющие

- Качественная современная криптография
- Ротация ключей – неограниченно долгое соединение
- Сверх компактный заголовок
- 0-RTT при некоторых условиях
- Широкий набор шифров – ГОСТ, Shang Mi
- Размер контекста 700B – 4KB, зависит от шифра
- SNI, application protocol discovery

# DTLS – недостатки

Ничего особо страшного

- Сложный запутанный конечный автомат handshake
- Пробелы в стандарте для многих его переходов
- Не описаны/не решены проблемы stateless reset

# DTLS – как воспользоваться

Взять готовое либо реализовать

# DTLS – как воспользоваться

Взять готовое либо реализовать

Нам нужна версия 1.3, это радикально улучшенная 1.2

# DTLSv1.3 – как воспользоваться

Взять готовое либо реализовать

- Отличная платная реализация на C (WolfSSL)
- OpenSSL кривоват, но улучшается
- Есть реализация на rust (прототип)
- Нет реализаций на Go (может, оно и к лучшему)

# DTLSv1.3 – как воспользоваться

Взять готовое либо реализовать

- Отличная платная реализация на C (WolfSSL)
- OpenSSL кривоват, но улучшается
- Есть реализация на rust (прототип)
- Нет реализаций на Go (может, оно и к лучшему)

Я сделал прототип, вышло 8000 строк, и ещё добавится

# VK DTLS

Реализация логического уровня VK RPC

# VK DTLS

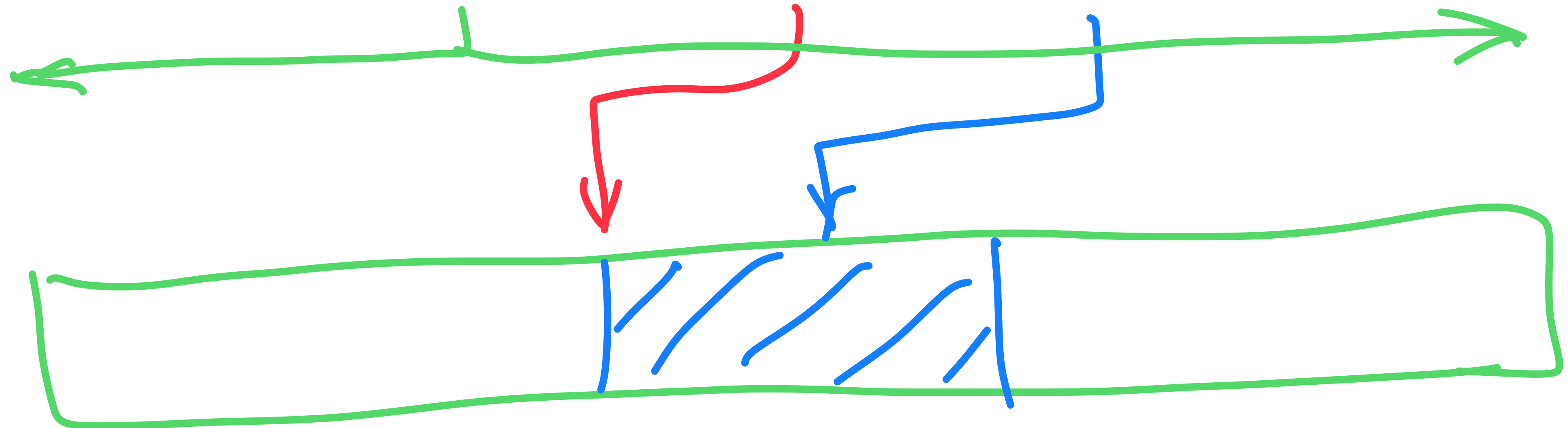
Реализация логического уровня VK RPC

- Достаточно простой протокол поверх DTLS
- Не пользуется никакими примитивами VK UDP
- Компактная структура данных
- Поддержка streaming в формате (если понадобится)

# VK DTLS

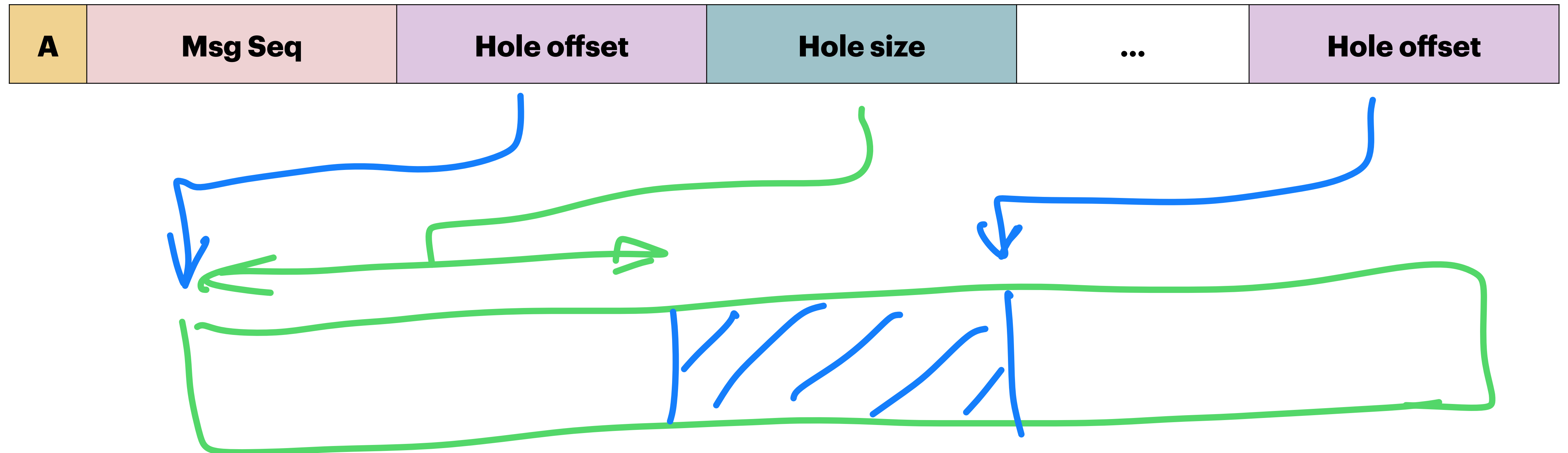
Сообщения/потоки с растущим номером, формат фрагмента

| M | Msg Seq | Msg Size | Fragment offset | Size | B | O | D | Y | . |
|---|---------|----------|-----------------|------|---|---|---|---|---|
|---|---------|----------|-----------------|------|---|---|---|---|---|



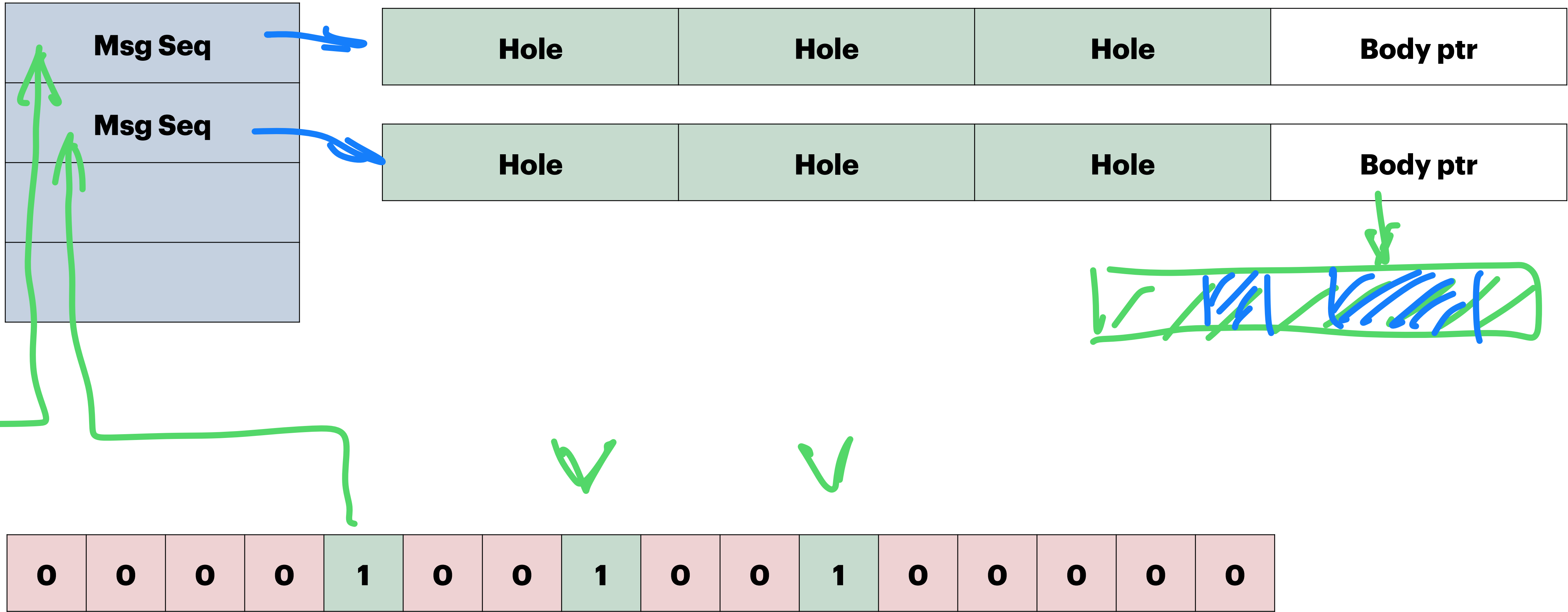
# VK DTLS

Подтверждения логического уровня, формат подтверждения



# VK DTLS

Пересборка сообщений



# VK DTLS

## Структура данных в соединении

- фиксированное число пересобираемых сообщений (8)
- Окно приёма на 64 новых номера сообщений
- фиксированное число отправляемых сообщений (8)
- Очередь на отправку (связный список)

# VK DTLS

## Структура данных в соединении

- фиксированное число пересобираемых сообщений (8)
- Окно приёма на 64 новых номера сообщений
- фиксированное число отправляемых сообщений (8)
- Очередь на отправку (связный список)

160 байтов на соединение (+DTLS context)

# VK DTLS

## Структура данных в транспорте

- Ограниченное количество соединений (100\_000)
- Ограниченное количество DTLS handshake (10\_000)
- Ограниченное число пересобираемых сообщений (100\_100)
- Ограниченное число отправляемых сообщений (100\_100)
- Ограниченный суммарный размер тел сообщений

# VK DTLS

## Структура данных в транспорте

- Ограниченное количество соединений (100\_000)
- Ограниченное количество DTLS handshake (10\_000)
- Ограниченное число пересобираемых сообщений (100\_100)
- Ограниченное число отправляемых сообщений (100\_100)
- Ограниченный суммарный размер тел сообщений

Полностью лимитированный транспорт

# Вопросы?

Григорий Бутейко [vk:@hrissan](https://vk.com/hrissan)